complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + foreach() printing

filter() null checks + foreach() printing

filter() comparisons + foreach() printing

filter() method calls returning boolean + foreach() printing

distinct() rely on equals() / hashcode() + foreach() printing

distinct() immutable objects + foreach() printing

map() transform objects + foreach() printing

map() create new objects + foreach() printing

map() call pure functions + foreach() printing

flatmap() return stream + foreach() printing

flatmap() split data + foreach() printing

flatmap() expand collections + foreach() printing

sorted() natural ordering + foreach() printing

sorted() custom comparator + foreach() printing

sorted() multi-field compare + foreach() printing

peek() logging + foreach() printing

peek() debugging + foreach() printing

peek() metrics + foreach() printing

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **One clear intermediate operation** is demonstrated
- **forEach() is the terminal operation**
- **Purpose = Printing output (not logging, not collecting)**
- Java **8+ compatible**
- Clean, interview-ready, minimal noise

---

1 filter() boolean checks + forEach() Printing

```
import java.util.*;

public class FilterBooleanForEachPrint {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      numbers.stream()
            .filter(n -> n % 2 == 0)
            .forEach(System.out::println);
   }
}
```

---

2 filter() null checks + forEach() Printing

```
import java.util.*;

public class FilterNullForEachPrint {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      names.stream()
            .filter(Objects::nonNull)
            .forEach(System.out::println);
   }
}
```

---

3 filter() comparisons + forEach() Printing

```
import java.util.*;

public class FilterComparisonForEachPrint {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      salaries.stream()
            .filter(s -> s >= 30000)
            .forEach(System.out::println);
   }
}
```

---

4 filter() method call returning boolean + forEach() Printing

```
import java.util.*;

public class FilterMethodForEachPrint {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Ravi", false)
      );

      users.stream()
            .filter(User::isActive)
            .forEach(u -> System.out.println(u.name));
   }
}
```

---

5 distinct() rely on equals() / hashCode() + forEach() Printing

```
import java.util.*;

public class DistinctEqualsForEachPrint {

    static class Person {
        String name;

        Person(String name) { this.name = name; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        people.stream()
                .distinct()
                .forEach(p -> System.out.println(p.name));
    }
}
```

---

6 distinct() immutable objects + forEach() Printing

```
import java.util.*;

public class DistinctImmutableForEachPrint {
    public static void main(String[] args) {
        List<String> languages = List.of("Java", "Java", "Python");

        languages.stream()
                .distinct()
```

```
            .forEach(System.out::println);
    }
}
```

---

7 map() transform objects + forEach() Printing

```
import java.util.*;

public class MapTransformForEachPrint {
    public static void main(String[] args) {
        List<String> names = List.of("anurag", "amit");

        names.stream()
            .map(String::toUpperCase)
            .forEach(System.out::println);
    }
}
```

---

8 map() create new objects + forEach() Printing

```
import java.util.*;

public class MapNewObjectForEachPrint {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Amit");

        names.stream()
            .map(UserDTO::new)
            .forEach(dto -> System.out.println(dto.name));
    }
}
```

---

9 map() call pure functions + forEach() Printing

```
import java.util.*;

public class MapPureFunctionForEachPrint {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      numbers.stream()
            .map(MapPureFunctionForEachPrint::square)
            .forEach(System.out::println);
   }
}
```

---

10 flatMap() return Stream + forEach() Printing

```
import java.util.*;

public class FlatMapStreamForEachPrint {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      lists.stream()
            .flatMap(List::stream)
            .forEach(System.out::println);
   }
}
```

---

11 flatMap() split data + forEach() Printing

```
import java.util.*;

public class FlatMapSplitForEachPrint {
   public static void main(String[] args) {
      List<String> sentences = List.of("Java Stream", "Flat Map");
      sentences.stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .forEach(System.out::println);
   }
}
```

---

12 flatMap() expand collections + forEach() Printing

```
import java.util.*;

public class FlatMapExpandForEachPrint {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
      );

      data.values().stream()
            .flatMap(Collection::stream)
            .forEach(System.out::println);
   }
}
```

---

13 sorted() natural ordering + forEach() Printing

```
import java.util.*;

public class SortedNaturalForEachPrint {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      numbers.stream()
            .sorted()
            .forEach(System.out::println);
   }
}
```

---

14 sorted() custom comparator + forEach() Printing

```
import java.util.*;

public class SortedCustomForEachPrint {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      names.stream()
            .sorted(Comparator.reverseOrder())
            .forEach(System.out::println);
   }
}
```

---

15 sorted() multi-field compare + forEach() Printing

```
import java.util.*;

public class SortedMultiFieldForEachPrint {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> employees = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      );

      employees.stream()
            .sorted(Comparator
                  .comparing((Employee e) -> e.name)
                  .thenComparingInt(e -> e.age))
            .forEach(e -> System.out.println(e.name + " " + e.age));
   }
}
```

---

16 peek() logging + forEach() Printing

```
import java.util.*;

public class PeekLoggingForEachPrint {
   public static void main(String[] args) {
      List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek: " + n))
            .forEach(System.out::println);
   }
}
```

---

17 peek() debugging + forEach() Printing

```
import java.util.*;

public class PeekDebugForEachPrint {
   public static void main(String[] args) {
      List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) System.out.println("Debug hit");
            })
            .forEach(System.out::println);
   }
}
```

---

18 peek() metrics + forEach() Printing

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsForEachPrint {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .forEach(System.out::println);

      System.out.println("Total processed: " + counter.get());
   }
}
```